Buổi 3

Các mở rộng của C++ so với C (phần 2)

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Hàm inline

1.1 Hàm inline – khái niệm và so sánh với macro
1.2 Khi nào dùng hàm inline?

2. Giá trị tham số mặc định

2.1 Tham số mặc định – cú pháp và quy tắc
2.2 Ứng dụng và lỗi thường gặp với tham số mặc định

3. Phép tham chiếu

3.1 Biến tham chiếu
3.2 Truyền tham số bằng tham chiếu
3.3 Tham chiếu hằng và hàm trả về nhiều giá trị
3.4 Hàm trả về tham chiếu
3.5 So sánh con trỏ và tham chiếu

4. Đa năng hóa hàm (Function Overloading)

4.1 Khái niệm đa năng hóa hàm
4.2 Quy tắc chọn hàm khi đa năng hóa
4.3 Ví dụ tổng hợp: bộ hàm hoán vị và tìm max đa năng hóa
4.4 Cơ chế bên trong và lưu ý khi đa năng hóa

5. Tổng kết và thực hành

5.1 Tóm tắt và bài thực hành buổi 3
5.2 Câu hỏi và bài tập

1.1 Hàm inline – khái niệm và so sánh với macro

- **Chi phí gọi hàm:** lưu địa chỉ trở về, đẩy tham số vào ngăn xếp, nhảy tới hàm, trả kết quả… Với hàm rất ngắn (1–2 lệnh) gọi nhiều lần trong vòng lặp, chi phí này đáng kể.

Hàm inline

Hàm khai báo với từ khóa **inline** là lời **đề nghị** trình biên dịch **thay lời gọi hàm bằng chính thân hàm** tại chỗ gọi ⇒ không tốn chi phí gọi hàm, nhưng vẫn giữ mọi ưu điểm của hàm (kiểm tra kiểu, tham số được tính một lần).

```
inline int BinhPhuong(int a) { return a * a; }
#define BP(a) a*a                  // macro của C
int main()
{  cout << BinhPhuong(2 + 3) << " "
        << BP(2 + 3) << endl;
}
```

```
25 11
```

- **Vì sao macro cho 11?** Tiền xử lý thay **văn bản**: `BP(2 + 3)` ⇒ `2 + 3*2 + 3` = 2 + 6 + 3 = **11**. Hàm inline tính tham số **trước**: (2 + 3) = 5 ⇒ 25.

Tiêu chí	Macro `#define`	Hàm inline
Xử lý bởi	Tiền xử lý (thay văn bản)	Trình biên dịch
Kiểm tra kiểu tham số	Không	Có
Tính tham số	Có thể tính **nhiều lần**	Đúng **một** lần
Lỗi do thiếu ngoặc	Thường gặp	Không
Gỡ lỗi	Khó	Như hàm thường

Ví dụ 3.1 – Tác dụng phụ của macro

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int i = 5, j = 3;
int m = MAX(i++, j);   // i++ được tính HAI lần ⇒ i = 7, m
= 6
```

- Với `inline int Max(int a, int b)` ⇒ i++ tính một lần: m = 5, i = 6.

1.2 Khi nào dùng hàm inline?

- **Nên dùng** cho hàm:
  - **Ngắn** (1–3 lệnh), gọi **nhiều lần** (trong vòng lặp).
  - Ví dụ: hàm truy cập `getX()`, `setX()` của lớp, Max, Min, BinhPhuong, GiaTriTuyetDoi.
- **Không nên / không thể** inline:
  - Hàm dài: mỗi chỗ gọi chép nguyên thân hàm ⇒ **phình to** mã máy.
  - Hàm **đệ quy**, hàm chứa vòng lặp phức tạp, switch lớn – trình biên dịch thường **bỏ qua** đề nghị inline.
- `inline` chỉ là **đề nghị**; trình biên dịch hiện đại tự quyết định (có thể inline cả hàm không khai báo inline khi tối ưu `-O2`).

- **Cú pháp và vị trí đặt:**
  - Định nghĩa hàm inline **trước** khi gọi (thường đặt trong tập tin .h) vì trình biên dịch cần thấy thân hàm tại chỗ gọi.
  - Hàm thành viên **định nghĩa ngay trong khai báo lớp** tự động được xem là inline (buổi 4).

Ví dụ 3.2 – Hàm inline và hàm thường

```
inline double DienTichTron(double r)
{  return 3.14159 * r * r;  }
inline int LaSoChan(int n) { return n % 2 == 0; }
int main()
{  double tong = 0;
   for (int r = 1; r <= 1000000; r++)
      tong += DienTichTron(r);    // không có 10⁶ lần gọi
hàm
}
```

- Sau khi trình biên dịch thay thế, vòng lặp trở thành `tong += 3.14159 * r * r;`.

2.1 Tham số mặc định – cú pháp và quy tắc

- C++ cho phép khai báo **giá trị mặc định** cho tham số; khi gọi hàm có thể **bỏ qua** các đối số đó và hàm tự dùng giá trị mặc định.

```
void InDong(char c = '-', int n = 10)
{  for (int i = 0; i < n; i++) cout << c;
   cout << endl;
}
int main()
{  InDong();          // c = '-', n = 10
   InDong('*');       // c = '*', n = 10
   InDong('=', 5);    // c = '=', n = 5
}
```

```
----------
**********
=====
```

- **Quy tắc:**
  - Các tham số có giá trị mặc định phải nằm ở **cuối** danh sách (bên phải): `void F(int a, int b = 1, int c = 2);` ✓, `void F(int a = 1, int b);` ✗.
  - Khi gọi chỉ được bỏ đối số **từ phải sang trái**: không thể bỏ c mà giữ n (`InDong(, 5)` ✗).
  - Giá trị mặc định ghi **một lần**, thường ở **khai báo nguyên mẫu** (prototype); định nghĩa hàm không ghi lại:

```
void InDong(char c = '-', int n = 10);   // prototype
...
void InDong(char c, int n) { ... }       // định nghĩa
```

- Giá trị mặc định có thể là hằng, biến toàn cục, hoặc biểu thức.

2.2 Ứng dụng và lỗi thường gặp với tham số mặc định

Ví dụ 3.3 – Một hàm thay cho nhiều hàm

```
double TinhLuong(double luongCB, double heSo = 1.0,
                 double phuCap = 0)
{  return luongCB * heSo + phuCap;  }
TinhLuong(5000000);              // 5.000.000
TinhLuong(5000000, 2.34);        // 11.700.000
TinhLuong(5000000, 2.34, 500000);// 12.200.000
```

Ví dụ 3.4 – Hàm tạo ngày với năm mặc định là năm hiện tại

```
void DatNgay(int d, int m, int y = 2026);
DatNgay(5, 10);          // 5/10/2026
DatNgay(30, 4, 1975);    // 30/4/1975
```

Lỗi	Ví dụ	Thông báo của trình biên dịch
Mặc định không ở cuối	`void F(int a = 1, int b);`	default argument missing for parameter 2
Ghi lại mặc định ở định nghĩa	prototype và định nghĩa đều có `= 10`	default argument given for parameter … redefinition
Nhập nhằng với đa năng hóa	`void h(int a, int b = 0); void h(int a);` rồi gọi `h(1)`	call of overloaded 'h(int)' is **ambiguous**

- **Lỗi nhập nhằng** (đã chạy thử với g++): `h(1, 2)` gọi được h(int, int); nhưng `h(1)` khớp **cả hai** hàm ⇒ lỗi biên dịch. ⇒ Không kết hợp tham số mặc định với một bản đa năng hóa có ít tham số hơn.

Tham chiếu (reference)

Tham chiếu là một **tên khác (bí danh)** của một biến đã có. Khai báo: `Kiểu &tenThamChieu = bien;`. Mọi thao tác trên tham chiếu chính là thao tác trên biến gốc.

```
int x = 1;
int &r = x;         // r là bí danh của x
r = 100;            // x = 100
cout << "x = " << x << endl;          // x = 100
cout << &x << " " << &r << endl;      // CÙNG một địa chỉ
```

```
      x, r
    ┌─────┐
    │ 100 │   địa chỉ 0x7ffd…10
    └─────┘
(một ô nhớ, hai tên)
```

- **Quy tắc:**
  - Phải **khởi tạo ngay** khi khai báo: `int &r;` ⇒ error: 'r' declared as reference but **not initialized**.
  - Sau khi khởi tạo, tham chiếu **không thể** đổi sang biến khác: `r = y;` là **gán giá trị** của y cho x, không phải đổi r.
  - Không có tham chiếu tới NULL, không có mảng các tham chiếu.

Ví dụ 3.5 – Tham chiếu và con trỏ

```
int a = 5, b = 9;
int &r = a;   int *p = &a;
r = b;        // a = 9 (r vẫn là a)
p = &b;       // p đổi sang trỏ b
*p = 0;       // b = 0
cout << a << " " << b;   // 9 0
```

Ví dụ 3.6 – Ba cách viết hàm hoán vị

```
void Swap1(int a, int b)          // truyền giá trị
{  int t = a; a = b; b = t;  }
void Swap2(int *a, int *b)        // truyền con trỏ (C)
{  int t = *a; *a = *b; *b = t;  }
void Swap3(int &a, int &b)        // truyền tham chiếu (C++)
{  int t = a; a = b; b = t;  }
int main()
{  int x = 1, y = 2;
   Swap1(x, y);  cout << x << " " << y << endl;
   Swap2(&x, &y); cout << x << " " << y << endl;
   Swap3(x, y);  cout << x << " " << y << endl;
}
```

```
1 2
2 1
1 2
```

- **Swap1:** a, b là **bản sao** của x, y ⇒ hoán vị bản sao, x, y **không đổi** (1 2).
- **Swap2:** truyền **địa chỉ**; hàm sửa qua `*a`, `*b` ⇒ hoán vị được (2 1). Cú pháp rườm rà: `&x` khi gọi, `*a` trong hàm; có nguy cơ truyền NULL.
- **Swap3:** a, b là **bí danh** của x, y ⇒ hoán vị được (1 2 trở lại); cú pháp **gọn** như truyền giá trị.

Cách truyền	Hàm sửa được biến gọi?	Sao chép dữ liệu?
Giá trị	Không	Có (tốn với struct lớn)
Con trỏ	Có	Không (chỉ chép địa chỉ)
Tham chiếu	Có	Không

3.3 Tham chiếu hằng và hàm trả về nhiều giá trị

- **Truyền struct/đối tượng lớn:** truyền giá trị phải **sao chép** cả khối dữ liệu; truyền tham chiếu thì không – nhưng hàm lại sửa được dữ liệu.
- ⇒ Dùng **tham chiếu hằng** `const Kiểu &`: không sao chép **và** không sửa được.

```
struct SinhVien { char HoTen[50]; float Diem[20]; };  // ≈
130 byte
void In1(SinhVien sv);          // sao chép 130 byte mỗi lần
gọi
void In2(const SinhVien &sv);   // chỉ truyền "địa chỉ", chỉ
đọc
void In2(const SinhVien &sv)
{  cout << sv.HoTen;
   // sv.Diem[0] = 10;          // LỖI: sv là const
}
```

- **Quy tắc thực hành:** kiểu cơ sở (int, double) ⇒ truyền giá trị; struct/lớp chỉ đọc ⇒ `const &`; cần sửa ⇒ `&`.

Ví dụ 3.7 – Hàm "trả về" nhiều kết quả qua tham chiếu

```
void ThongKe(const int a[], int n,
             int &Min, int &Max, double &TB)
{  Min = Max = a[0];  long S = 0;
   for (int i = 0; i < n; i++)
   {  if (a[i] < Min) Min = a[i];
      if (a[i] > Max) Max = a[i];
      S += a[i];
   }
   TB = double(S) / n;
}
int a[] = {3, 8, 1, 9, 4};  int mn, mx;  double tb;
ThongKe(a, 5, mn, mx, tb);
cout << mn << " " << mx << " " << tb;   // 1 9 5
```

- Tham chiếu tới con trỏ `int *&p` cho phép hàm cấp phát và **trả lại** con trỏ mới (khắc phục lỗi ví dụ 2.13 buổi 2).

3.4 Hàm trả về tham chiếu

- Hàm trả về **tham chiếu** ⇒ lời gọi hàm là một **biến** (lvalue): có thể đặt **bên trái phép gán**.

```
int M[5] = {3, 8, 1, 9, 4};
int &PhanTu(int i) { return M[i]; }
int main()
{  PhanTu(2) = 50;            // gán vào M[2]
   for (int i = 0; i < 5; i++)
      cout << M[i] << ' ';
}
```

```
3 8 50 9 4
```

- Đây là nền tảng của việc đa năng hóa toán tử `[]` (`a[i] = 5` với lớp mảng – buổi 8) và toán tử `<<` trả về `ostream&` để viết liên tiếp `cout << a << b`.

Lỗi nghiêm trọng: trả về tham chiếu tới biến cục bộ

```
int &Sai()
{  int x = 5;
   return x;      // x bị hủy khi hàm kết thúc!
}
```

g++ cảnh báo: `warning: reference to local variable 'x' returned`. Tham chiếu trả về trỏ vào vùng nhớ **đã giải phóng** ⇒ giá trị rác hoặc crash.

- **Chỉ trả về tham chiếu tới:** biến toàn cục, biến static, phần tử của mảng/đối tượng **tồn tại sau khi hàm kết thúc**, tham số truyền bằng tham chiếu.
- Buổi 5 sẽ thấy: hàm thành viên trả về tham chiếu tới dữ liệu **private** sẽ **phá vỡ** tính đóng gói ⇒ cần cẩn thận.

3.5 So sánh con trỏ và tham chiếu

Tiêu chí	Con trỏ `int *p`	Tham chiếu `int &r`
Bản chất	Biến chứa **địa chỉ**	**Bí danh** của biến
Khởi tạo	Không bắt buộc (nguy cơ con trỏ rác)	**Bắt buộc**
Giá trị NULL	Có thể	Không
Đổi đối tượng tham chiếu	Được (`p = &b`)	Không
Truy cập giá trị	`*p`, `p->x`	`r`, `r.x` (như biến thường)
Số học địa chỉ	`p + 1`, `p++`	Không
Cấp phát động, mảng động, DSLK	Bắt buộc dùng con trỏ	Không dùng được
Truyền tham số để sửa	Được – cú pháp rườm rà	**Được – gọn, an toàn**

Ví dụ 3.9 – Chọn cái nào?

- Hàm cần sửa đối số, đối số chắc chắn tồn tại ⇒ **tham chiếu**: `void Tang(int &x)`.
- Đối số có thể "không có" (tùy chọn) ⇒ **con trỏ** và kiểm tra NULL: `void InLoi(const char *ghiChu = NULL)`.
- Duyệt mảng, danh sách liên kết, cấp phát động ⇒ **con trỏ**.
- Truyền đối tượng lớn chỉ để đọc ⇒ **const &**.

Ví dụ 3.10 – Dự đoán kết quả

```
int a = 5, b = 9;
int &r = a;
int *p = &a;
r = b;      // gán giá trị 9 cho a
p = &b;     // p trỏ sang b
*p = 0;     // b = 0
cout << a << " " << b;
```

```
9 0
```

- Lỗi hiểu nhầm thường gặp: nghĩ `r = b;` làm r "trỏ sang" b ⇒ dự đoán sai a = 5.

Đa năng hóa hàm

C++ cho phép định nghĩa **nhiều hàm cùng tên** trong cùng phạm vi, miễn là **danh sách tham số khác nhau** (số lượng hoặc kiểu hoặc thứ tự kiểu). Khi gọi, trình biên dịch **tự chọn** hàm phù hợp với các đối số – đây là một dạng **đa hình lúc biên dịch**.

```
int Max(int a, int b)
{  cout << "[int] ";    return a > b ? a : b;  }
double Max(double a, double b)
{  cout << "[double] "; return a > b ? a : b;  }
int Max(int a, int b, int c)
{  cout << "[3 int] ";  return Max(Max(a, b), c);  }
int main()
{  cout << Max(3, 7) << endl;
   cout << Max(2.5, 1.5) << endl;
   cout << Max(4, 9, 6) << endl;
}
```

```
[int] 7
[double] 2.5
[3 int] [int] [int] 9
```

- Trong C phải đặt tên khác nhau: `MaxInt`, `MaxDouble`, `Max3Int`… ⇒ khó nhớ.
- Dòng 3: Max 3 tham số gọi lại Max 2 tham số **hai lần** ⇒ in "[int]" hai lần.
- **Ví dụ trong thư viện:** `abs(int)`, `abs(double)`; `cout <<` in được int, double, char*… chính là đa năng hóa toán tử `<<`.

Kiểu trả về KHÔNG dùng để phân biệt

```
int F(int a);
double F(int a);   // error: ambiguating new declaration
```

Khi gọi `F(3);` trình biên dịch không thể biết chọn hàm nào ⇒ hai hàm chỉ khác kiểu trả về **không** được phép đa năng hóa.

4.2 Quy tắc chọn hàm khi đa năng hóa

- Trình biên dịch so khớp đối số với các hàm cùng tên theo **thứ tự ưu tiên**:

Mức	Loại khớp	Ví dụ
1	**Khớp chính xác** (kể cả mảng → con trỏ, T → const T)	`f(5)` khớp `f(int)`
2	**Nâng cấp** (promotion): char, short, bool → int; float → double	`f('A')` → `f(int)`; `f(3.5f)` → `f(double)`
3	**Chuyển đổi chuẩn**: int ↔ double, int → long…	`g(5)` với g(long), g(double)
4	Chuyển đổi do người dùng định nghĩa (constructor, toán tử chuyển kiểu)	buổi 8

- Nếu ở mức thấp nhất tìm được có **từ hai hàm trở lên** khớp ngang nhau ⇒ lỗi **ambiguous** (nhập nhằng).

Ví dụ 3.8 – Chạy thử với g++

```
void f(int x)    { cout << "f(int)\n"; }
void f(double x) { cout << "f(double)\n"; }
void g(long x)   { cout << "g(long)\n"; }
void g(double x) { cout << "g(double)\n"; }
f(5);  f(5.0);  f('A');  f(3.5f);
g(5);
```

```
f(int)        ← khớp chính xác
f(double)     ← khớp chính xác
f(int)        ← nâng cấp char → int
f(double)     ← nâng cấp float → double
error: call of overloaded 'g(int)' is ambiguous
```

- `g(5)`: int → long và int → double đều là **chuyển đổi chuẩn** (cùng mức 3) ⇒ nhập nhằng. Sửa: `g(5L)` hoặc `g(long(5))`.

4.3 Ví dụ tổng hợp: bộ hàm hoán vị và tìm max đa năng hóa

```
#include <iostream>
#include <cstring>
using namespace std;
inline void HoanVi(int &a, int &b)
{  int t = a; a = b; b = t;  }
inline void HoanVi(double &a, double &b)
{  double t = a; a = b; b = t;  }
void HoanVi(char *a, char *b)          // chuỗi
{  char t[100];
   strcpy(t, a); strcpy(a, b); strcpy(b, t);
}
int Max(const int a[], int n)          // max của mảng
{  int m = a[0];
   for (int i = 1; i < n; i++) if (a[i] > m) m = a[i];
   return m;
}
const char *Max(const char *s, const char *t)
{  return strcmp(s, t) > 0 ? s : t;  }
```

```
int main()
{  int x = 1, y = 2;      HoanVi(x, y);
   double u = 1.5, v = 2.5; HoanVi(u, v);
   char s[100] = "An", t[100] = "Binh";
   HoanVi(s, t);
   cout << x << y << " " << u << " " << v
        << " " << s << " " << t << endl;
   int a[] = {3, 8, 1, 9, 4};
   cout << Max(a, 5) << " "
        << Max("Hoa", "Lan") << endl;
}
```

```
21 2.5 1.5 Binh An
9 Lan
```

- Cùng tên **HoanVi** cho 3 kiểu, cùng tên **Max** cho mảng và chuỗi; buổi 11: **template** thay thế việc viết lặp.

4.4 Cơ chế bên trong và lưu ý khi đa năng hóa

- **Name mangling:** trình biên dịch C++ **mã hóa** tên hàm kèm kiểu tham số thành tên duy nhất trong mã máy. Xem bằng lệnh `nm` (g++):

```
$ g++ -c nm.cpp && nm nm.o | grep Max
_Z3Maxii      ← int Max(int, int)
_Z3Maxdd      ← double Max(double, double)
_Z3Maxiii     ← int Max(int, int, int)
```

- Vì vậy các hàm cùng tên vẫn **phân biệt** được ở mức liên kết; kiểu trả về **không** nằm trong tên mã hóa ⇒ không thể đa năng hóa chỉ theo kiểu trả về.
- Gọi hàm C từ C++ phải khai báo `extern "C"` để tắt mã hóa tên.

- **Những khác biệt KHÔNG đủ để đa năng hóa:**

Hai khai báo	Kết quả
`void F(int a); void F(int b);`	Chỉ khác tên tham số ⇒ trùng
`int F(int); double F(int);`	Chỉ khác kiểu trả về ⇒ lỗi
`void F(int a); void F(const int a);`	const ở tham số giá trị ⇒ trùng
`void F(int *p); void F(int p[]);`	Mảng = con trỏ ⇒ trùng

- **Hợp lệ:** `void F(int &a); void F(const int &a);` (khác nhau thật sự).
- **Lời khuyên thiết kế:** các hàm cùng tên phải làm **cùng một việc** về ý nghĩa (Max, In, DienTich…); tránh trộn tham số mặc định với đa năng hóa gây nhập nhằng.

5.1 Tóm tắt và bài thực hành buổi 3

Tính năng	Cú pháp	Tác dụng chính
Hàm inline	`inline int F(int a) {…}`	Giảm chi phí gọi hàm ngắn, an toàn hơn macro
Tham số mặc định	`void F(int a, int b = 0);`	Gọi hàm với ít đối số hơn
Tham chiếu	`int &r = x;`	Bí danh; truyền tham số không sao chép, sửa được biến gọi
Tham chiếu hằng	`const T &x`	Không sao chép, không sửa
Trả về tham chiếu	`int &F();`	Lời gọi hàm đặt bên trái phép gán
Đa năng hóa hàm	Cùng tên, khác tham số	Một tên cho nhiều kiểu dữ liệu

Bài thực hành – Bộ hàm hoán vị, tìm max đa năng hóa và dùng tham chiếu

- 1) Viết các hàm `HoanVi` cho int, double, char, chuỗi, struct PhanSo (dùng tham chiếu); kiểm thử bằng main.
- 2) Viết các hàm `Max` đa năng hóa: 2 số nguyên, 3 số nguyên, 2 số thực, mảng số nguyên, mảng số thực, 2 phân số; mỗi hàm in ra tên phiên bản được gọi như ví dụ 3.8.
- 3) Viết hàm `void SapXep(int a[], int n, bool tang = true)` (tham số mặc định) dùng HoanVi.
- 4) Viết hàm `int &PhanTuLonNhat(int a[], int n)` trả về **tham chiếu** tới phần tử lớn nhất; dùng nó để gán phần tử lớn nhất bằng 0.
- 5) Thử tạo ra lỗi `ambiguous` và giải thích; thử viết macro bình phương và so sánh với hàm inline cho `BP(i++)`.

5.2 Câu hỏi và bài tập

Câu hỏi

- 1) Ưu điểm của hàm inline so với macro? Khi nào không nên dùng inline?
- 2) Nêu quy tắc khai báo và gọi hàm có tham số mặc định.
- 3) Phân biệt truyền tham số bằng giá trị, con trỏ, tham chiếu, tham chiếu hằng.

Bài tập dự đoán kết quả (có đáp số)

```
void F(int &a, int b) { a += b; b += a; }
int &G(int &x) { x *= 2; return x; }
int main()
{  int p = 1, q = 2;
   F(p, q);       cout << p << " " << q << endl;
   G(p) = 10;     cout << p << endl;
   G(G(q));       cout << q << endl;
}
```

→ **Đáp số:** `3 2` / `10` / `8`. (F: a là tham chiếu ⇒ p = 3; b là bản sao ⇒ q không đổi. G(p) nhân đôi p rồi trả về chính p ⇒ gán 10. G(G(q)) nhân đôi q hai lần: 2 → 8.)

Bài tập lập trình

- 1) Viết hàm `GiaiPTBac2(double a, double b, double c, double &x1, double &x2)` trả về số nghiệm.
- 2) Viết các hàm đa năng hóa `DienTich` cho hình tròn (r), chữ nhật (a, b), tam giác (a, b, c).